HDOC DAY-55

Write a C program that reads the size and elements of a variable-length integer array, arr[], and an integer k such that 0 < k ≤ size. Pass the array to a function named window_maximum() using its address. Use pointers to pass the array between the calling function and window_maximum(). The function window_maximum() must:

1. Consider every contiguous subarray of size k as a sliding window.
2. Find the maximum element in each window while moving from left to right.
3. Display the maximum elements in the order of their corresponding windows.
4. Separate the maximum elements by spaces.
5. Display -1 if k is invalid.
6. Use a nested-loop approach to solve the problem.

Example 1:

Array: 1 3 -1 -3 5 3 6 7

k: 3

Output: 3 3 5 5 6 7

Example 2:

Array: 1 2 3 4 5

k: 2

Output: 2 3 4 5

Example 3:

Array: 1 2 3

k: 4

Output: -1

Algorithm

1. Start.
2. Read the size of the array.

3. Read all array elements using pointers.
4. Read the window size k.
5. Pass the array to window_maximum() using its address.
6. If k <= 0 or k > size, print -1.
7. Otherwise, use an outer loop to move the window from left to right.
8. For each window, assume its first element is the maximum.
9. Use an inner loop to compare all k elements and find the maximum.
10. Print the maximum of each window separated by spaces.
11. Stop.

C Program:

```
#include <stdio.h>

void window_maximum(int *arr, int size, int k)

{

  // Check for invalid k

  if (k <= 0 || k > size)

  {

    printf("-1");

    return;

  }


  // Move the sliding window from left to right

  for (int i = 0; i <= size - k; i++)

  {

    int max = *(arr + i);


    // Find maximum element in the current window

    for (int j = 1; j < k; j++)
```

```
        {
            if (*(arr + i + j) > max)
            {
                max = *(arr + i + j);
            }
        }

        printf("%d", max);

        if (i < size - k)
            printf(" ");
    }
}

int main()
{
    int size, k;

    printf("Enter size: ");
    scanf("%d", &size);

    int arr[size];

    printf("Enter array elements: ");
    for (int i = 0; i < size; i++)
    {
        scanf("%d", arr + i);
    }
```

```
    printf("Enter k: ");
    scanf("%d", &k);

    printf("Output: ");
    window_maximum(arr, size, k);

    return 0;
}
```

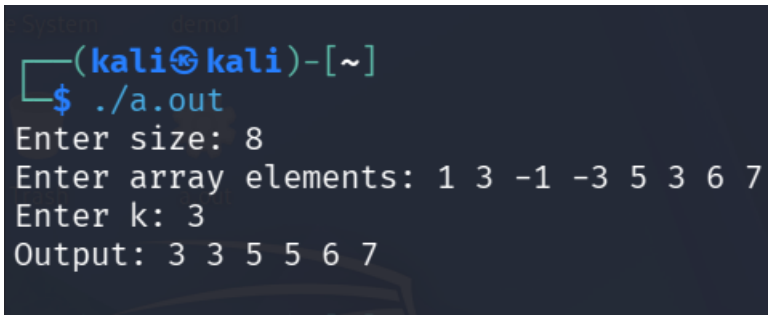
Test Case Table:

Test Case	Array	k	Output
1	1 3 -1 -3 5 3 6 7	3	3 3 5 5 6 7
2	1 2 3 4 5	2	2 3 4 5
3	1 2 3	4	-1
4	5 4 3 2 1	1	5 4 3 2 1
5	2 8 4 6 1	5	8

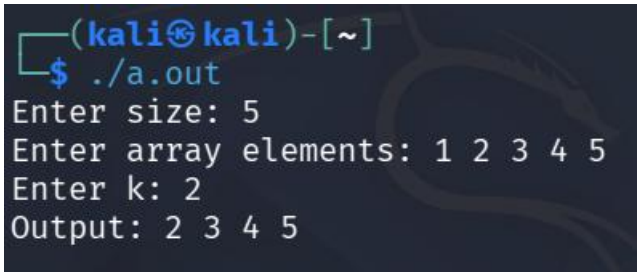
Compilation and Execution:

The following program has been compiled and executed successfully.

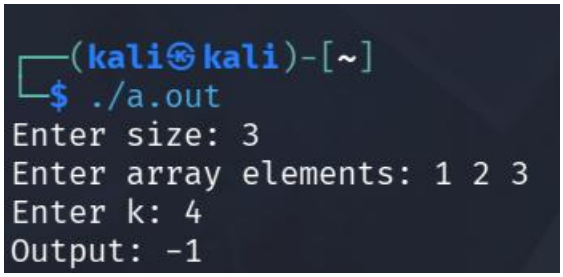
Test Case 1:



Test Case 2:



Test Case 3:



Test Case 4:



Test Case 5:

